

Mastering LangChain

A Complete Developer's Guide

From First Principles to Production-Grade AI Applications

Sabarinathan J

Senior AI / ML Engineer

6 Parts • 19 Chapters • 70+ Concepts • 50+ Code Examples

Architecture Diagrams • Interview Prep • Production Patterns

Prerequisites: Python • Basic LLM Knowledge

Preface

LangChain has rapidly become the most widely adopted framework for building LLM-powered applications. Whether you are building a simple chatbot, a retrieval-augmented question-answering system, or a fully autonomous multi-agent pipeline, LangChain provides composable building blocks to do it reliably and at scale.

This book takes you from zero LangChain knowledge to expert-level mastery. It assumes you are comfortable writing Python and have a conceptual understanding of how large language models work. Every concept is built up from scratch with theory, architecture diagrams, and complete working code examples validated against LangChain v0.3.

✓ Tip

All code targets: langchain>=0.3, langchain-core>=0.3, langchain-openai>=0.2, langchain-community>=0.3. Install: `pip install langchain langchain-openai langchain-community`

— Sabarinathan J

Table of Contents

PART I — Foundations

- | | |
|------|--|
| Ch 1 | LangChain Architecture & Ecosystem
Packages, abstractions, Runnable protocol |
| Ch 2 | LLM & Chat Model Interfaces
Providers, config, streaming, caching |
| Ch 3 | Prompts & Prompt Templates
PromptTemplate, ChatPrompt, few-shot, Hub |
| Ch 4 | Output Parsers
Pydantic, JSON, structured output, custom |

PART II — The Composition Layer

- | | |
|------|--|
| Ch 5 | LangChain Expression Language (LCEL)
Pipe operator, Runnable primitives, streaming |
| Ch 6 | Memory & Stateful Conversations
Buffer, summary, entity, persistent backends |

PART III — Data & Retrieval

- | | |
|------|---|
| Ch 7 | Document Loaders & Text Splitters
20+ loaders, chunking strategies |
| Ch 8 | Embeddings & Vector Stores
OpenAI, HuggingFace, FAISS, Chroma, Pinecone |
| Ch 9 | Retrievers
Multi-query, compression, ensemble, self-query |

PART IV — RAG — Retrieval-Augmented Generation

- | | |
|-------|--|
| Ch 10 | Naive RAG Pipeline
Load→split→embed→store→retrieve→generate |
| Ch 11 | Advanced RAG Techniques
HyDE, multi-query, reranking |
| Ch 12 | Conversational RAG & Evaluation
History-aware retrieval, RAGAS metrics |

PART V — Agents & Tools

- | | |
|-------|---|
| Ch 13 | Chains — Legacy & Modern
LLMChain, map-reduce, LCEL equivalents |
| Ch 14 | Agent Fundamentals
ReAct, OpenAI Tools, AgentExecutor |

- | | |
|--------------|---|
| Ch 15 | Tools & Toolkits
Custom tools, SQL agent, error handling |
| Ch 16 | LangGraph & Multi-Agent Systems
StateGraph, checkpointers, supervisor |

PART VI — Production & Mastery

- | | |
|--------------|---|
| Ch 17 | Callbacks & LangSmith
Tracing, custom handlers, evaluation |
| Ch 18 | Caching, Deployment & Security
LangServe, Redis cache, security |
| Ch 19 | Interview Preparation Guide
60 questions: conceptual, design, debugging |

PART I

Foundations

Build a solid mental model of LangChain's architecture, package structure, and core abstractions before writing a single line of production code.

CHAPTER 1

LangChain Architecture & Ecosystem

Before writing any LangChain code you need a clear map of the ecosystem. This chapter explains what LangChain is, how its packages fit together, and introduces the Runnable protocol — the universal interface that ties everything together.

1.1 What Is LangChain?

LangChain is an open-source Python (and JavaScript) framework that provides composable abstractions for building LLM-powered applications. Created by Harrison Chase in October 2022, it has become the most widely used LLM framework.

The core insight is that real LLM applications require far more than an API call. You need to manage conversation context (memory), retrieve relevant information from your own data (RAG), chain multiple LLM calls (chains), give LLMs the ability to use external tools (agents), and observe what the model is doing (tracing). LangChain provides standardised building blocks for all of these concerns.

■ Key Concept

LangChain is not a replacement for the LLM API. It is a layer of abstractions on top of LLM APIs (OpenAI, Anthropic, etc.) that makes complex application patterns easy to build, test, and maintain.

1.2 Package Ecosystem

Package	Purpose & Contents
langchain-core	Base abstractions: Runnable, BasePromptTemplate, BaseOutputParser. Zero heavy dependencies. The foundation.
langchain	High-level chains, agents, memory classes. Depends on langchain-core.
langchain-community	Third-party integrations: 50+ vector stores, 100+ document loaders, tools.
langchain-openai	ChatOpenAI, OpenAIEmbeddings, AzureChatOpenAI.
langchain-anthropic	ChatAnthropic — Claude models.
langchain-huggingface	HuggingFace Hub and local pipeline models.
langgraph	Stateful multi-agent orchestration. StateGraph, checkpointers.
langsmith	Observability SDK for tracing, testing, and evaluation.

1.3 Architecture Diagram

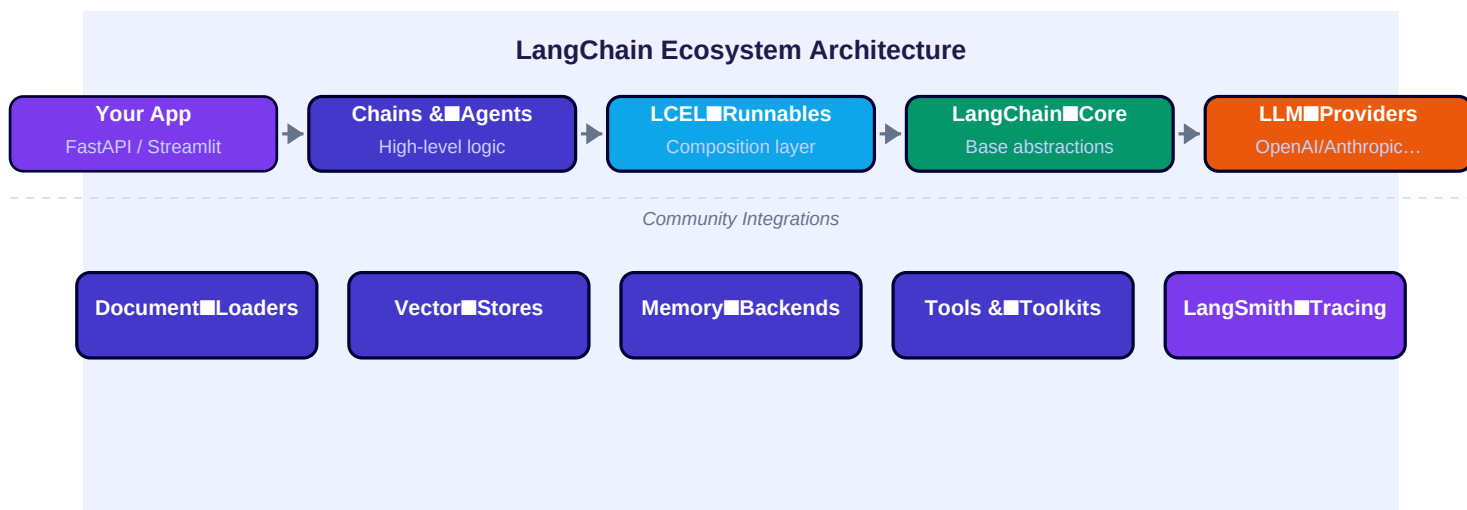


Figure 1.1 — LangChain ecosystem layers and community integrations

1.4 The Runnable Protocol

Every component in modern LangChain implements the Runnable interface from langchain-core. This gives every component the same four invocation methods, making them infinitely composable.

■ Key Concept

The Runnable protocol is the single most important abstraction in LangChain. Once you understand it, every other component becomes predictable.

Method	Description
<code>invoke(input, config=None)</code>	Synchronous single call. Returns one output.
<code>stream(input, config=None)</code>	Synchronous generator. Yields output chunks as they arrive.
<code>batch(inputs, config=None)</code>	Parallel execution. Returns list of outputs.
<code>ainvoke</code> / <code>astream</code> / <code>abatch</code>	Async equivalents. Use in FastAPI or asyncio programs.

■ `runnable_protocol.py`

```

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# 1. invoke -- synchronous single call
response = llm.invoke('What is LangChain?')
print(response.content)

# 2. stream -- receive tokens as they arrive
for chunk in llm.stream('Explain RAG in one sentence.'):
    print(chunk.content, end='', flush=True)

# 3. batch -- parallel calls
questions = ['What is RAG?', 'What is an agent?', 'What is LCEL?']
responses = llm.batch(questions)
for q, r in zip(questions, responses):
    print(f'Q: {q[:30]} -> {r.content[:60]}')

# 4. async invoke
import asyncio

async def main():
    result = await llm.ainvoke('Hello async world!')
    print(result.content)

asyncio.run(main())

```

1.5 Installation & Project Setup

■ installation.sh

```

# Install core packages
pip install langchain langchain-core langchain-openai langchain-community

# For LangSmith observability
pip install langsmith

# Verify installation
python -c 'import langchain; print(langchain.__version__)'

```

■ .env

```

# .env -- NEVER commit this file to git
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
LANGCHAIN_API_KEY=ls_... # LangSmith tracing key
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=my-project

```

■ load_env.py

```

# At the top of every script / notebook
from dotenv import load_dotenv
load_dotenv() # reads .env automatically

import os
print(os.environ['OPENAI_API_KEY'][:8], '...') # verify loaded

```

■ Warning

Never hardcode API keys in source code. Always load from .env via python-dotenv and add .env to .gitignore.

CHAPTER 2

LLM & Chat Model Interfaces

LangChain provides a unified interface to dozens of LLM providers. This chapter covers the difference between LLM and ChatModel types, how to configure every important parameter, streaming, caching, and cost tracking.

2.1 LLM vs ChatModel

Aspect	LLM (BaseLLM)	ChatModel (BaseChatModel)
Input	Plain string	List of BaseMessage objects
Output	Plain string	AIMessage (with .content, .response_metadata)
Examples	OpenAI text-davinci, older GPT-3	ChatOpenAI, ChatAnthropic, ChatGoogleGenerativeAI
Use today	Legacy — avoid for new code	Preferred — all modern models are ChatModels

BaseMessage Types

■ message_types.py

```
from langchain_core.messages import (
    HumanMessage,      # role='user'      -- from the human
    AIMessage,         # role='assistant' -- from the model
    SystemMessage,     # role='system'  -- instructions / persona
    ToolMessage,       # role='tool'    -- result of a tool call
)

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')

messages = [
    SystemMessage(content='You are a helpful Python tutor.'),
    HumanMessage(content='What is a list comprehension?'),
]

response = llm.invoke(messages)
print(type(response))      # <class 'AIMessage'>
print(response.content)    # explanation text
print(response.response_metadata) # token usage, finish_reason
```

2.2 Provider Integrations

■ providers.py

```
# OpenAI (most common)
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.7, max_tokens=1024)

# Anthropic Claude
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model='claude-3-5-sonnet-20241022', temperature=0)

# Google Gemini
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model='gemini-1.5-flash')

# Ollama (local, free, no API key)
from langchain_ollama import ChatOllama
llm = ChatOllama(model='llama3.2', temperature=0)

# HuggingFace Hub
from langchain_huggingface import HuggingFaceEndpoint
llm = HuggingFaceEndpoint(
    repo_id='mistralai/Mistral-7B-Instruct-v0.3',
    max_new_tokens=512,
)

# Azure OpenAI
from langchain_openai import AzureChatOpenAI
llm = AzureChatOpenAI(
    azure_deployment='gpt-4o',
    azure_endpoint='https://YOUR_RESOURCE.openai.azure.com/',
    api_version='2024-08-01-preview',
)
```

2.3 Configuration Parameters

Parameter	Effect & When to Use
temperature	0 = deterministic, 1 = creative. Use 0 for facts, 0.7+ for creative writing.
max_tokens	Cap response length. Always set in production to control cost.
top_p	Nucleus sampling. Don't set both top_p and temperature simultaneously.
frequency_penalty	Reduce repetition (OpenAI only, range -2 to 2).
stop	Strings that stop generation. E.g. stop=['\n\n', 'END']
timeout	Seconds before raising timeout error. Essential in production.
max_retries	Auto-retries on rate-limit / server errors. Default is 2.

2.4 Streaming

■ streaming.py

```

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini', streaming=True)

# Sync streaming
for chunk in llm.stream('Summarise LangChain in 3 sentences.'):
    print(chunk.content, end='', flush=True)
print() # newline at end

# Async streaming
import asyncio

async def stream_async():
    async for chunk in llm.astream('What is RAG?'):
        print(chunk.content, end='', flush=True)

asyncio.run(stream_async())

```

2.5 Caching and Cost Tracking

■ caching_and_cost.py

```

# Response caching
from langchain.globals import set_llm_cache
from langchain_community.cache import InMemoryCache, SQLiteCache

set_llm_cache(InMemoryCache()) # dev -- resets on restart
set_llm_cache(SQLiteCache('.langchain.db')) # staging -- persists to disk

# Redis distributed cache (production, multi-instance)
import redis
from langchain_community.cache import RedisCache
set_llm_cache(RedisCache(
    redis=redis.Redis.from_url('redis://localhost:6379')
))

# Token usage & cost tracking
from langchain_community.callbacks import get_openai_callback
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')

with get_openai_callback() as cb:
    result = llm.invoke('Explain RAG vs fine-tuning.')
    print(result.content[:100])

print(f'Prompt tokens:      {cb.prompt_tokens}')
print(f'Completion tokens: {cb.completion_tokens}')
print(f'Total cost (USD):    ${cb.total_cost:.6f}')

```

2.6 Fallbacks and Retry

■ fallbacks_retry.py

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic

primary = ChatOpenAI(model='gpt-4o', timeout=10, max_retries=1)
fallback = ChatAnthropic(model='claude-3-haiku-20240307')
cheap = ChatOpenAI(model='gpt-4o-mini')

# If primary fails -> tries fallback -> then cheap
robust_llm = primary.with_fallbacks([fallback, cheap])

# Retry with exponential back-off on transient errors
resilient_llm = primary.with_retry(
    stop_after_attempt=3,
    wait_exponential_jitter=True,
)

response = robust_llm.invoke('What is LangChain?')
print(response.content)
```

CHAPTER 3

Prompts & Prompt Templates

Prompts are the primary interface between your application and the LLM. LangChain's prompt abstractions create reusable, parameterised, composable templates that are easy to version, test, and share.

3.1 PromptTemplate (String-based)

■ prompt_template.py

```
from langchain_core.prompts import PromptTemplate

# from_template factory -- recommended
template = PromptTemplate.from_template(
    'You are a {role}. Answer the question: {question}'
)

# format() returns a plain string
prompt_str = template.format(
    role='Python expert',
    question='What is a decorator?'
)
print(prompt_str)
# -> 'You are a Python expert. Answer the question: What is a decorator?'

# Partial variables -- pre-fill some variables ahead of time
partial_tmpl = template.partial(role='data scientist')
print(partial_tmpl.format(question='What is a p-value?'))
```

3.2 ChatPromptTemplate

■ chat_prompt_template.py

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# Tuple shorthand: (role, template_string)
chat_template = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert {domain} tutor. Be concise.'),
    ('human', 'Explain {concept} with a simple example.'),
])

# format_messages() returns list[BaseMessage]
messages = chat_template.format_messages(
    domain='machine learning',
    concept='gradient descent',
)
for msg in messages:
    print(f'{msg.type}: {msg.content[:80]}')

# Use directly in an LCEL chain
chain = chat_template | ChatOpenAI(model='gpt-4o-mini') | StrOutputParser()
print(chain.invoke({'domain': 'Python', 'concept': 'decorators'}))
```

3.3 MessagesPlaceholder

■ messages_placeholder.py

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage
from langchain_openai import ChatOpenAI

# MessagesPlaceholder injects a dynamic list of messages (history)
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder(variable_name='history'), # dynamic slot
    ('human', '{input}'),
])

history = [
    HumanMessage(content='My name is Kavi.'),
    AIMessage(content='Nice to meet you, Kavi!'),
]

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
response = llm.invoke(
    prompt.format_messages(history=history, input='What is my name?')
)
print(response.content) # -> 'Your name is Kavi.'
```

3.4 Few-Shot Prompting

■ few_shot_prompting.py

```
from langchain_core.prompts import (
    FewShotChatMessagePromptTemplate,
    ChatPromptTemplate,
)
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

examples = [
    {'input': 'happy', 'output': 'sad'},
    {'input': 'tall', 'output': 'short'},
    {'input': 'energetic', 'output': 'lethargic'},
]

example_template = ChatPromptTemplate.from_messages([
    ('human', '{input}'),
    ('ai', '{output}'),
])

few_shot_prompt = FewShotChatMessagePromptTemplate(
    examples=examples,
    example_prompt=example_template,
)

final_prompt = ChatPromptTemplate.from_messages([
    ('system', 'Give the antonym of every input word.'),
    few_shot_prompt,
    ('human', '{input}'),
])

chain = final_prompt | ChatOpenAI(model='gpt-4o-mini') | StrOutputParser()
print(chain.invoke({'input': 'beautiful'})) # -> 'ugly'
```

3.5 LangChain Hub

■ langchain_hub.py

```
from langchain import hub

# Pull a community prompt (no login for public prompts)
rag_prompt = hub.pull('rlm/rag-prompt')
print(rag_prompt.messages[0].prompt.template[:200])

# Save locally as YAML
from langchain_core.prompts import ChatPromptTemplate, load_prompt

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a {role}. Answer in {language}.'),
    ('human', '{question}'),
])
prompt.save('my_prompt.yaml')
loaded = load_prompt('my_prompt.yaml')
```

CHAPTER 4

Output Parsers

LLMs output raw text. Output parsers transform that text into structured Python objects that your application can reliably work with.

4.1 StrOutputParser

■ str_output_parser.py

```
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# Extracts .content from AIMessage, returns plain Python str
chain = (
    ChatPromptTemplate.from_template('Tell me a fact about {animal}')
    | ChatOpenAI(model='gpt-4o-mini')
    | StrOutputParser()
)

result = chain.invoke({'animal': 'octopus'})
print(type(result)) # <class 'str'>
print(result)       # 'Octopuses have three hearts...'
```

4.2 PydanticOutputParser

■ pydantic_output_parser.py

```
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
from typing import List

class MovieReview(BaseModel):
    title: str = Field(description='Title of the movie')
    rating: float = Field(description='Rating 0.0 to 10.0')
    pros: List[str] = Field(description='Positive aspects')
    cons: List[str] = Field(description='Negative aspects')
    verdict: str = Field(description='One-sentence verdict')

parser = PydanticOutputParser(pydantic_object=MovieReview)

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a film critic. {format_instructions}'),
    ('human', 'Review the movie: {movie}'),
]).partial(format_instructions=parser.get_format_instructions())

chain = prompt | ChatOpenAI(model='gpt-4o-mini') | parser
review = chain.invoke({'movie': 'Inception (2010)'})

print(type(review)) # <class 'MovieReview'>
print(review.title) # 'Inception'
print(review.rating) # 9.2
print(review.pros) # ['Stunning visuals', 'Complex plot', ...]
```

4.3 with_structured_output -- The Modern Way

■ Key Concept

`with_structured_output()` uses native function/tool calling for near-100% format compliance. Prefer this over `PydanticOutputParser` for any model that supports tool calling (GPT-4, Claude 3, Gemini 1.5).

■ with_structured_output.py

```
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
from typing import List, Optional

class PersonInfo(BaseModel):
    name: str = Field(description='Full name')
    age: Optional[int] = Field(description='Age if mentioned')
    occupation: str = Field(description='Job or role')
    skills: List[str] = Field(description='Technical skills')

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# No format instructions needed in prompt!
structured_llm = llm.with_structured_output(PersonInfo)

result = structured_llm.invoke(
    'Sabarinathan is a 32-year-old ML engineer who knows Python, PyTorch, LangChain.'
)
print(result.name)      # 'Sabarinathan'
print(result.age)       # 32
print(result.skills)    # ['Python', 'PyTorch', 'LangChain']
```

4.4 OutputFixingParser

■ output_fixing_parser.py

```
from langchain.output_parsers import OutputFixingParser
from langchain_core.output_parsers import PydanticOutputParser
from langchain_openai import ChatOpenAI
from pydantic import BaseModel

class Actor(BaseModel):
    name: str
    film_names: list[str]

base_parser = PydanticOutputParser(pydantic_object=Actor)

# If parsing fails, sends error back to LLM and asks it to fix output
fixing_parser = OutputFixingParser.from_llm(
    parser=base_parser,
    llm=ChatOpenAI(model='gpt-4o-mini'),
)

# Simulate malformed output
bad_output = '{"name: Tom Hanks, films: ['Forrest Gump']}]'"
result = fixing_parser.parse(bad_output)
print(result.name)      # 'Tom Hanks'
print(result.film_names) # ['Forrest Gump']
```

PART II

The Composition Layer

Learn LCEL — LangChain Expression Language — the pipe-based composition system that makes complex multi-step AI pipelines simple, readable, and streaming-by-default. Then master memory.

CHAPTER 5

LangChain Expression Language (LCEL)

LCEL is the most important concept in modern LangChain. It is a declarative composition system built on the Runnable protocol. Every new LangChain application should be written in LCEL.

■ Key Concept

LCEL = Runnable Protocol + Pipe Operator (|) + Composable Primitives. Streaming works end-to-end by default with no extra code.

5.1 The Pipe Operator |

■ lcel_pipe_operator.py

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

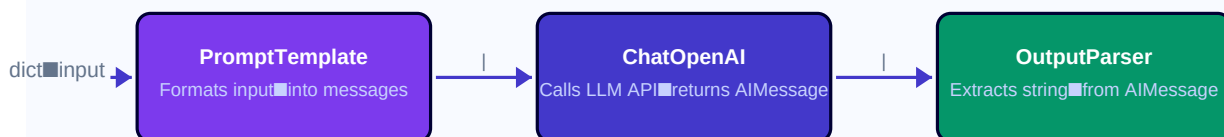
# The canonical LCEL chain: prompt | llm | parser
chain = (
    ChatPromptTemplate.from_template(
        'What is {topic}? Answer in 2 sentences.'
    )
    | ChatOpenAI(model='gpt-4o-mini', temperature=0)
    | StrOutputParser()
)

# invoke -- synchronous
print(chain.invoke({'topic': 'LCEL'}))

# stream -- tokens appear as they arrive
for chunk in chain.stream({'topic': 'RAG'}):
    print(chunk, end='', flush=True)

# batch -- parallel execution
results = chain.batch([
    {'topic': 'LangChain'},
    {'topic': 'Vector databases'},
    {'topic': 'Embeddings'},
])
for r in results:
    print(r[:80])
```

LCEL Pipeline: prompt | llm | parser



Each step is a Runnable. The | operator chains them. Data flows left to right.

Figure 5.1 — LCEL data flow: each step is a Runnable, | chains them left to right

5.2 Runnable Primitives

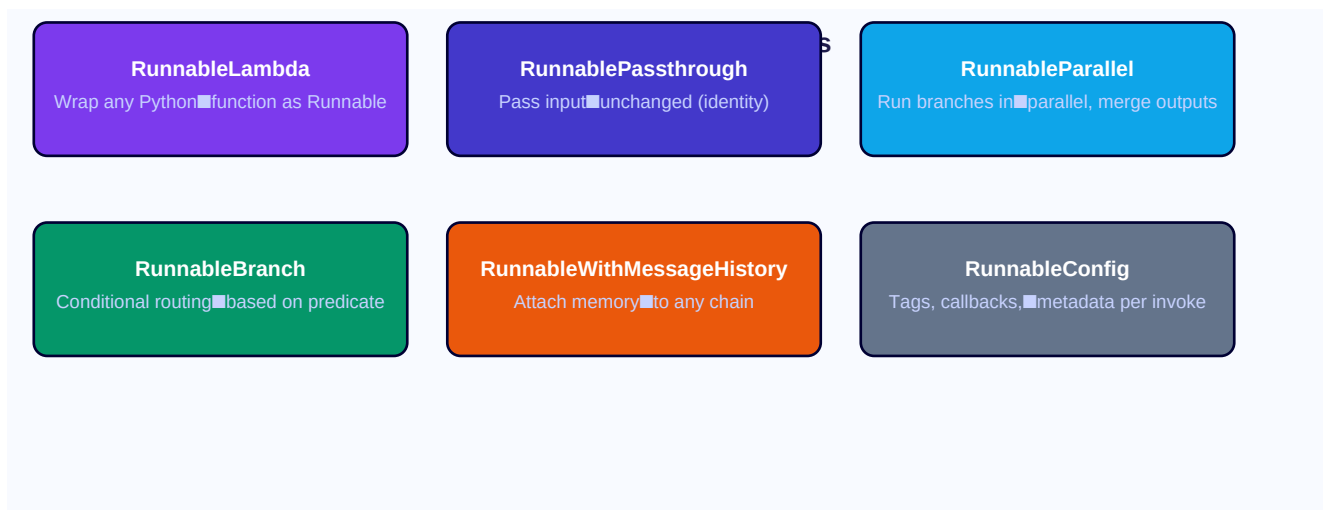


Figure 5.2 — LCEL Runnable Primitives Overview

RunnableLambda -- Wrap Any Python Function

■ runnable_lambda.py

```
from langchain_core.runnables import RunnableLambda
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

def word_counter(text: str) -> dict:
    return {'text': text, 'word_count': len(text.split())}

chain = (
    ChatPromptTemplate.from_template('Summarise: {topic}')
    | ChatOpenAI(model='gpt-4o-mini')
    | StrOutputParser()
    | RunnableLambda(word_counter) # any Python function
)

result = chain.invoke({'topic': 'machine learning'})
print(result['word_count']) # number of words in summary
print(result['text'][:80]) # the summary text
```

RunnableParallel -- Parallel Branch Execution

■ runnable_parallel.py

```

from langchain_core.runnables import RunnableParallel
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# Both branches execute in parallel on the SAME input
parallel_chain = RunnableParallel({
    'sentiment': (
        ChatPromptTemplate.from_template(
            'Sentiment (positive/negative/neutral): {review}'
        ) | llm | StrOutputParser()
    ),
    'summary': (
        ChatPromptTemplate.from_template(
            'Summarise in 10 words: {review}'
        ) | llm | StrOutputParser()
    ),
})

result = parallel_chain.invoke(
    {'review': 'The product is amazing but very expensive!'}
)
print(result['sentiment']) # 'positive'
print(result['summary'])  # 'Amazing product but overpriced'

```

RunnableBranch -- Conditional Routing

■ runnable_branch.py

```

from langchain_core.runnables import RunnableBranch
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o-mini')

python_chain = (
    ChatPromptTemplate.from_template(
        'Answer this Python question: {question}'
    ) | llm | StrOutputParser()
)
sql_chain = (
    ChatPromptTemplate.from_template(
        'Answer this SQL question: {question}'
    ) | llm | StrOutputParser()
)
general_chain = (
    ChatPromptTemplate.from_template('Answer: {question}')
    | llm | StrOutputParser()
)

branch = RunnableBranch(
    (lambda x: 'python' in x['question'].lower(), python_chain),
    (lambda x: 'sql' in x['question'].lower(), sql_chain),
    general_chain, # default if no condition matches
)

print(branch.invoke({'question': 'How do Python decorators work?'}))
print(branch.invoke({'question': 'How do I write a SQL JOIN?'}))

```

5.3 Fallback, Retry, and Configurable

■ fallback_retry_configurable.py

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import ConfigurableField

prompt = ChatPromptTemplate.from_template('Answer: {question}')
primary = ChatOpenAI(model='gpt-4o', timeout=10, max_retries=1)
backup = ChatOpenAI(model='gpt-4o-mini')

# Fallback: if primary fails -> backup
robust = prompt | primary.with_fallbacks([backup]) | StrOutputParser()

# Retry with exponential back-off
retry_chain = (
    prompt
    | primary.with_retry(stop_after_attempt=3,
                       wait_exponential_jitter=True)
    | StrOutputParser()
)

# Runtime-configurable model swap
llm = ChatOpenAI(model='gpt-4o-mini').configurable_alternatives(
    ConfigurableField(id='llm'),
    default_key='gpt4mini',
    claude=ChatAnthropic(model='claude-3-haiku-20240307'),
)
chain = prompt | llm | StrOutputParser()

# Use default
r1 = chain.invoke({'question': 'What is LCEL?'})

# Switch to Claude at runtime -- zero code change
r2 = chain.with_config(
    configurable={'llm': 'claude'}
).invoke({'question': 'What is LCEL?'})
```

CHAPTER 6

Memory & Stateful Conversations

LLMs are stateless: they have no memory of previous turns unless you provide history explicitly. This chapter covers every memory type and how to integrate persistent backends for production multi-turn chatbots.

6.1 Memory Types

LangChain Memory Types — Comparison			
 Buffer	Stores every message verbatim	Unlimited	Full fidelity
 Window	Keeps last K turns only	K turns	Loses old context
 Summary	LLM compresses history to summary	Bounded	May lose detail
 Entity	Tracks named entities + their facts	Per entity	Structured
 Vector	Semantic long-term store	All history	Approximate recall

Figure 6.1 — LangChain memory types comparison

6.2 RunnableWithMessageHistory

```
■ runnable_with_history.py
```

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_openai import ChatOpenAI

# In-memory session store
store = {}

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder(variable_name='history'),
    ('human', '{input}'),
])

chain = prompt | ChatOpenAI(model='gpt-4o-mini')

chain_with_memory = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key='input',
    history_messages_key='history',
)

config = {'configurable': {'session_id': 'user_001'}}

r1 = chain_with_memory.invoke(
    {'input': 'My name is Sabarinathan.'}, config=config
)
print(r1.content)

r2 = chain_with_memory.invoke(
    {'input': 'What is my name?'}, config=config
)
print(r2.content) # 'Your name is Sabarinathan.'
```

6.3 Persistent Backends

■ persistent_memory_backends.py

```
# Redis (production, multi-user, distributed)
from langchain_community.chat_message_histories import RedisChatMessageHistory

def get_redis_history(session_id: str):
    return RedisChatMessageHistory(
        session_id=session_id,
        url='redis://localhost:6379',
        ttl=3600, # expire after 1 hour of inactivity
    )

# PostgreSQL (SQL-backed)
from langchain_community.chat_message_histories import SQLChatMessageHistory

def get_sql_history(session_id: str):
    return SQLChatMessageHistory(
        session_id=session_id,
        connection_string='postgresql://user:pass@localhost/chatdb',
    )

# Just swap get_session_history in RunnableWithMessageHistory
chain_with_redis = RunnableWithMessageHistory(
    chain,
    get_redis_history, # <-- only this line changes
    input_messages_key='input',
    history_messages_key='history',
)
```

6.4 ConversationSummaryBufferMemory

For long conversations, storing every message verbatim fills the context window. Summary memory uses an LLM to compress older turns into a concise summary, keeping only the most recent turns verbatim.

■ summary_buffer_memory.py

```
from langchain.memory import ConversationSummaryBufferMemory
from langchain_openai import ChatOpenAI

summarizer = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# Keep last 500 tokens verbatim; summarise anything older
memory = ConversationSummaryBufferMemory(
    llm=summarizer,
    max_token_limit=500,
    return_messages=True,
    memory_key='chat_history',
)

memory.save_context(
    {'input': 'I am building a RAG system for legal documents.'},
    {'output': 'What embedding model are you using?'},
)
memory.save_context(
    {'input': 'Using text-embedding-3-large from OpenAI.'},
    {'output': 'Good choice. Chunking by paragraph is recommended.'},
)

vars = memory.load_memory_variables({})
for msg in vars['chat_history']:
    print(f'{msg.type}: {msg.content[:100]}')
```

PART III

Data & Retrieval

Master the ingestion pipeline: loading documents from any source, splitting them intelligently, embedding them into vector space, and retrieving the most relevant chunks. This is the foundation of every RAG system.

CHAPTER 7

Document Loaders & Text Splitters

Before you can retrieve information, you must ingest it. This chapter covers loading documents from every major source and splitting them into retrieval-friendly chunks without losing meaning.

7.1 The Document Object

■ document_object.py

```
from langchain_core.documents import Document

# A Document is the atomic unit of the RAG pipeline
doc = Document(
    page_content='LangChain is a framework for building LLM apps.',
    metadata={
        'source': 'docs/intro.pdf',
        'page': 1,
        'author': 'Harrison Chase',
    },
)

print(doc.page_content)      # The text content
print(doc.metadata['source']) # 'docs/intro.pdf'

# All loaders return List[Document]
from langchain_community.document_loaders import TextLoader
loader = TextLoader('readme.txt', encoding='utf-8')
docs = loader.load()
print(len(docs))             # 1 for a text file
print(docs[0].metadata)      # {'source': 'readme.txt'}
```

7.2 File-Based Loaders

■ file_loaders.py

```
# PDF -- one Document per page
from langchain_community.document_loaders import PyPDFLoader
pages = PyPDFLoader('report.pdf').load()
print(f'{len(pages)} pages')
print(pages[0].metadata) # {'source': 'report.pdf', 'page': 0}

# PDF with complex layouts / tables
from langchain_community.document_loaders import UnstructuredPDFLoader
elements = UnstructuredPDFLoader('complex.pdf', mode='elements').load()

# Word .docx
from langchain_community.document_loaders import Docx2txtLoader
docs = Docx2txtLoader('document.docx').load()

# CSV -- one Document per row
from langchain_community.document_loaders import CSVLoader
docs = CSVLoader(file_path='data.csv', encoding='utf-8').load()

# Python source code
from langchain_community.document_loaders.generic import GenericLoader
from langchain_community.document_loaders.parsers import LanguageParser
from langchain_text_splitters import Language

loader = GenericLoader.from_filesystem(
    './src', glob='**/*.py', suffixes=['.py'],
    parser=LanguageParser(language=Language.PYTHON),
)
code_docs = loader.load()
```

7.3 Web & Directory Loaders

■ web_and_directory_loaders.py

```
# Web page
from langchain_community.document_loaders import WebBaseLoader
import bs4

loader = WebBaseLoader(
    web_paths=['https://python.langchain.com/docs/introduction/'],
    bs_kwargs=dict(
        parse_only=bs4.SoupStrainer(class_=('post-content',))
    ),
)
docs = loader.load()

# Wikipedia
from langchain_community.document_loaders import WikipediaLoader
docs = WikipediaLoader(
    query='LangChain framework', load_max_docs=3
).load()

# Recursive URL crawler
from langchain_community.document_loaders import RecursiveUrlLoader
from bs4 import BeautifulSoup

loader = RecursiveUrlLoader(
    url='https://docs.python.org/3/',
    max_depth=2,
    extractor=lambda h: BeautifulSoup(h, 'lxml').get_text(separator=' '),
)
docs = loader.load()
print(f'Crawled {len(docs)} pages')

# Bulk directory -- all PDFs recursively
from langchain_community.document_loaders import DirectoryLoader
from langchain_community.document_loaders import PyPDFLoader

loader = DirectoryLoader(
    path='./knowledge_base/',
    glob='**/*.pdf',
    loader_cls=PyPDFLoader,
    show_progress=True,
    use_multithreading=True,
)
all_docs = loader.load()
print(f'Loaded {len(all_docs)} document pages')
```

7.4 Text Splitters

■ Key Concept

`chunk_size` and `chunk_overlap` are the two most impactful hyperparameters in RAG. Start with `chunk_size=1000`, `chunk_overlap=200` and tune based on your documents.

Splitter	Best For
<code>RecursiveCharacterTextSplitter</code>	General text — default choice. Splits on paragraph, newline, sentence, word.
<code>TokenTextSplitter</code>	Token-precise splitting (tiktoken). Use when near context limits.
<code>MarkdownHeaderTextSplitter</code>	Markdown — preserves section structure in metadata.
<code>CodeTextSplitter</code>	Source code — language-aware (Python, JS, Go, etc.).
<code>SemanticChunker</code>	Embedding-based boundary detection at semantic topic shifts.

■ text_splitters.py

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,      # target chars per chunk
    chunk_overlap=200,    # overlap between consecutive chunks
    add_start_index=True, # record char offset in metadata
)

from langchain_community.document_loaders import PyPDFLoader
docs = PyPDFLoader('paper.pdf').load()
chunks = splitter.split_documents(docs)

print(f'Pages: {len(docs)}')
print(f'Chunks: {len(chunks)}')
print(f'Chunk 0 length: {len(chunks[0].page_content)} chars')

# Token-based splitting
from langchain_text_splitters import TokenTextSplitter
token_splitter = TokenTextSplitter(
    chunk_size=256,      # 256 tokens per chunk
    chunk_overlap=50,
    encoding_name='cl100k_base', # GPT-4 tokeniser
)

# Semantic chunker
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

semantic_splitter = SemanticChunker(
    embeddings=OpenAIEmbeddings(),
    breakpoint_threshold_type='percentile',
    breakpoint_threshold_amount=95,
)
text = open('article.txt').read()
s_chunks = semantic_splitter.create_documents([text])
print(f'Semantic chunks: {len(s_chunks)}')
```

CHAPTER 8

Embeddings & Vector Stores

Embeddings convert text into dense numerical vectors capturing semantic meaning. Vector stores index these vectors for fast similarity search. Together they form the retrieval backbone of every RAG system.

8.1 Embedding Providers

■ Warning

Always use `embed_query()` for search-time queries and `embed_documents()` for indexing. Asymmetric models use different representations internally — mixing them degrades retrieval quality.

■ embeddings.py

```
# OpenAI (recommended for production)
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model='text-embedding-3-small') # 1536 dims

query_vec = embeddings.embed_query('What is gradient descent?')
print(len(query_vec)) # 1536

doc_vecs = embeddings.embed_documents([
    'Gradient descent is an optimisation algorithm.',
    'Neural networks learn by adjusting weights.',
])
print(len(doc_vecs[0])) # 1536

# HuggingFace (local, free, 384 dims)
from langchain_huggingface import HuggingFaceEmbeddings
local_emb = HuggingFaceEmbeddings(
    model_name='sentence-transformers/all-MiniLM-L6-v2',
    model_kwargs={'device': 'cpu'},
    encode_kwargs={'normalize_embeddings': True},
)

# Embedding cache -- avoids re-embedding identical text
from langchain.embeddings import CacheBackedEmbeddings
from langchain_community.storage import LocalFileStore

store = LocalFileStore('./emb_cache/')
cached = CacheBackedEmbeddings.from_bytes_store(
    embeddings, store, namespace=embeddings.model
)
# First call hits API; second call with same text is instant
```

8.2 Vector Stores

Vector Store	Type	Best For
FAISS	In-memory / local file	Prototyping, no infra required
Chroma	Local persistent	Single-machine dev / staging
Pinecone	Managed cloud	Production SaaS, serverless option

Weaviate	Self-hosted / cloud	Hybrid search (dense + BM25)
Qdrant	Self-hosted / cloud	High performance, rich metadata filtering
pgvector	PostgreSQL extension	Already using Postgres, unified data + vectors

■ vector_stores.py

```

from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document

embeddings = OpenAIEmbeddings(model='text-embedding-3-small')

docs = [
    Document(page_content='Python is a high-level language.',
              metadata={'source': 'wiki'}),
    Document(page_content='LangChain makes building LLM apps easy.',
              metadata={'source': 'docs'}),
    Document(page_content='RAG combines retrieval with generation.',
              metadata={'source': 'paper'}),
]

# Build vector store
vs = FAISS.from_documents(docs, embeddings)

# Similarity search
results = vs.similarity_search('How does retrieval work?', k=2)
for doc in results:
    print(doc.page_content)

# With relevance scores
scored = vs.similarity_search_with_relevance_scores(
    'What is LangChain?', k=3
)
for doc, score in scored:
    print(f'Score: {score:.4f} | {doc.page_content}')

# Persist and reload
vs.save_local('./faiss_index')
vs2 = FAISS.load_local(
    './faiss_index', embeddings,
    allow_dangerous_deserialization=True
)

# Chroma (persistent)
from langchain_chroma import Chroma
chroma_vs = Chroma(
    collection_name='my_docs',
    embedding_function=embeddings,
    persist_directory='./chroma_db',
)

# Search with metadata filter
results = chroma_vs.similarity_search(
    'Main conclusion?', k=3,
    filter={'source': 'report.pdf'},
)

```

CHAPTER 9

Retrievers

A Retriever is a Runnable that takes a string query and returns List[Document]. LangChain provides many retriever types beyond simple similarity search, each addressing a specific weakness in basic vector retrieval.

9.1 VectorStoreRetriever

■ vector_store_retriever.py

```
# Standard similarity retriever
retriever = vs.as_retriever(
    search_type='similarity',
    search_kwargs={'k': 4},
)
results = retriever.invoke('What is gradient descent?')

# MMR -- balance relevance AND diversity in results
mmr_retriever = vs.as_retriever(
    search_type='mmr',
    search_kwargs={'k': 4, 'fetch_k': 20, 'lambda_mult': 0.5},
)

# Score threshold -- only return above minimum similarity
threshold_retriever = vs.as_retriever(
    search_type='similarity_score_threshold',
    search_kwargs={'score_threshold': 0.75, 'k': 5},
)
```

9.2 MultiQueryRetriever

A single query misses relevant documents due to phrasing differences. MultiQueryRetriever generates multiple rephrased queries, retrieves for all, then deduplicates results. This significantly improves recall.

■ multi_query_retriever.py

```
from langchain.retrievers import MultiQueryRetriever
from langchain_openai import ChatOpenAI

base_retriever = vs.as_retriever(search_kwargs={'k': 3})

multi_retriever = MultiQueryRetriever.from_llm(
    retriever=base_retriever,
    llm=ChatOpenAI(model='gpt-4o-mini', temperature=0),
    include_original=True, # include the original query too
)

# Generates ~3 query variants, retrieves for each, deduplicates
results = multi_retriever.invoke(
    'What methods improve LLM retrieval accuracy?'
)
print(f'Retrieved {len(results)} unique documents')
```

9.3 EnsembleRetriever -- Hybrid Search

Dense vector search excels at semantic similarity but misses exact keyword matches. BM25 excels at exact terms. Combining both via Reciprocal Rank Fusion consistently outperforms either alone.

■ ensemble_retriever.py

```
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

# BM25 keyword retriever
bm25 = BM25Retriever.from_documents(docs, k=3)

# Dense semantic retriever
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

faiss_r = FAISS.from_documents(
    docs, OpenAIEmbeddings()
).as_retriever(search_kwargs={'k': 3})

# Ensemble using Reciprocal Rank Fusion
ensemble = EnsembleRetriever(
    retrievers=[bm25, faiss_r],
    weights=[0.5, 0.5],
)

results = ensemble.invoke('Python data structures')
for doc in results:
    print(doc.page_content[:80])
```

9.4 ParentDocumentRetriever

■ parent_document_retriever.py

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Embed small child chunks (precise), return large parent chunks (context)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=200)
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1000)

vectorstore = Chroma(
    collection_name='children',
    embedding_function=OpenAIEmbeddings()
)
docstore = InMemoryStore()

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=docstore,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)

from langchain_community.document_loaders import TextLoader
retriever.add_documents(TextLoader('article.txt').load())

results = retriever.invoke('What are the key findings?')
print(f'Parent chunk size: {len(results[0].page_content)} chars') # ~1000
```

9.5 SelfQueryRetriever

■ self_query_retriever.py

```
from langchain.retrievers.self_query.base import SelfQueryRetriever
from langchain.chains.query_constructor.base import AttributeInfo
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_core.documents import Document

docs = [
    Document(page_content='RAG improves factual accuracy.',
              metadata={'year': 2023, 'topic': 'RAG', 'rating': 8.5}),
    Document(page_content='LangGraph enables stateful agents.',
              metadata={'year': 2024, 'topic': 'agents', 'rating': 9.5}),
]
vs = Chroma.from_documents(docs, OpenAIEmbeddings())

metadata_field_info = [
    AttributeInfo(name='year', description='Publication year', type='integer'),
    AttributeInfo(name='topic', description='Topic area', type='string'),
    AttributeInfo(name='rating', description='Quality rating', type='float'),
]

retriever = SelfQueryRetriever.from_llm(
    llm=ChatOpenAI(model='gpt-4o-mini', temperature=0),
    vectorstore=vs,
    document_contents='AI research papers',
    metadata_field_info=metadata_field_info,
)

# LLM converts this to: semantic='agents' + filter: year > 2023
results = retriever.invoke('papers about agents published after 2023')
for doc in results:
    print(doc.metadata, '->', doc.page_content[:60])
```

PART IV

RAG — Retrieval-Augmented Generation

RAG is the most important LLM application pattern. It grounds responses in your own data, reducing hallucinations and enabling domain-specific knowledge.

CHAPTER 10

Naive RAG Pipeline

The simplest RAG pipeline: Load → Split → Embed → Store → Retrieve → Generate. Build this first, then improve. Every advanced RAG technique is an improvement to one of these six steps.

10.1 RAG Architecture

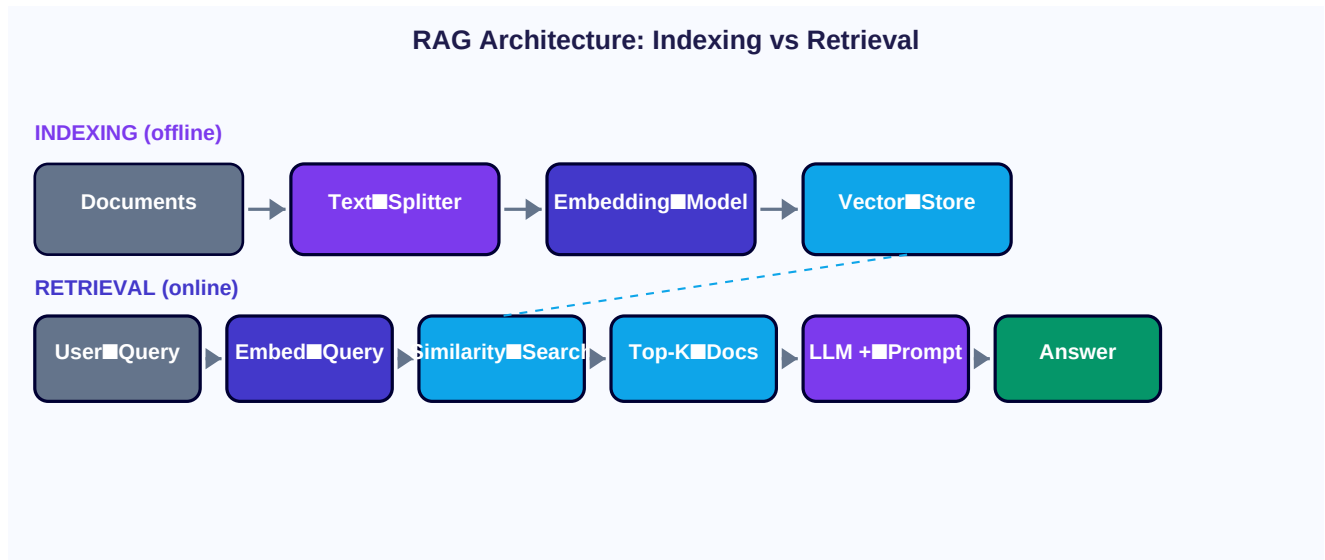


Figure 10.1 — RAG: Indexing phase (offline) and Retrieval phase (online)

10.2 Complete Naive RAG

■ naive_rag.py

```

from langchain_community.document_loaders import WebBaseLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableParallel
import bs4

# STEP 1: LOAD
loader = WebBaseLoader(
    web_paths=['https://lilianweng.github.io/posts/2023-06-23-agent/'],
    bs_kwargs=dict(
        parse_only=bs4.SoupStrainer(class_=('post-content',))
    ),
)
raw_docs = loader.load()

# STEP 2: SPLIT
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)
chunks = splitter.split_documents(raw_docs)
print(f'{len(chunks)} chunks created')

# STEP 3+4: EMBED + STORE
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vectorstore = FAISS.from_documents(chunks, embeddings)

# STEP 5: RETRIEVER
retriever = vectorstore.as_retriever(search_kwargs={'k': 4})

# STEP 6: GENERATE
def format_docs(docs):
    return '\n\n'.join(doc.page_content for doc in docs)

prompt = ChatPromptTemplate.from_messages([
    ('system',
     'Use the context to answer. Say so if the answer is not there.\n\n'
     'Context:\n{context}'),
    ('human', '{question}'),
])

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
rag_chain = (
    RunnableParallel({
        'context': retriever | format_docs,
        'question': RunnablePassthrough(),
    })
    | prompt | llm | StrOutputParser()
)

answer = rag_chain.invoke('What are the components of an LLM agent?')
print(answer)

```

10.3 RAG with Source Citations

■ rag_with_citations.py

```
from langchain_core.runnables import RunnableParallel

# Return answer AND source documents
rag_with_sources = RunnableParallel({
    'answer': rag_chain,
    'sources': retriever,
})

result = rag_with_sources.invoke('What is task decomposition?')
print('Answer:', result['answer'])
print('\nSources:')
for doc in result['sources']:
    src = doc.metadata.get('source', 'unknown')
    print(f' - {src} | {doc.page_content[:60]}...')
```

CHAPTER 11

Advanced RAG Techniques

Naive RAG has known failure modes: poor recall, hallucination, and inability to handle complex queries. This chapter covers HyDE, reranking, and contextual compression to address each failure mode.

11.1 HyDE -- Hypothetical Document Embeddings

Problem: query vectors and document vectors live in different embedding spaces. A question embedding is rarely close to an answer embedding. HyDE bridges this by generating a hypothetical answer, then using that answer as the retrieval query.

■ hyde_rag.py

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import (
    RunnableParallel, RunnablePassthrough, RunnableLambda
)

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.3)

# Step 1: generate hypothetical answer
hyde_prompt = ChatPromptTemplate.from_template(
    'Write a detailed technical paragraph answering: {question}\n'
    'Write as if you are the author of a reference document.'
)
hyde_chain = hyde_prompt | llm | StrOutputParser()

# Step 2: use the hypothetical answer as retrieval query
def hyde_retrieve(question: str):
    hypothetical = hyde_chain.invoke({'question': question})
    return retriever.invoke(hypothetical) # search with hypo doc

hyde_rag = (
    RunnableParallel({
        'context': RunnableLambda(hyde_retrieve) | format_docs,
        'question': RunnablePassthrough(),
    })
    | prompt | llm | StrOutputParser()
)

print(hyde_rag.invoke('What planning techniques do LLM agents use?'))
```

11.2 Reranking

Retrieve a large candidate set (top-20), then use a cross-encoder reranker to score each document against the query and return only the top-5. This dramatically improves precision.

■ reranking.py

```
# Cohere Rerank API (cloud, very accurate)
from langchain_cohere import CohereRerank
from langchain.retrievers import ContextualCompressionRetriever

wide_retriever = vectorstore.as_retriever(search_kwargs={'k': 20})

reranker = CohereRerank(model='rerank-english-v3.0', top_n=5)
reranking_retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=wide_retriever,
)

reranked_rag = (
    RunnableParallel({
        'context': reranking_retriever | format_docs,
        'question': RunnablePassthrough(),
    })
    | prompt
    | ChatOpenAI(model='gpt-4o-mini')
    | StrOutputParser()
)

# Local cross-encoder (no API cost)
from langchain.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder

local_model = HuggingFaceCrossEncoder(
    model_name='BAAI/bge-reranker-base'
)
local_reranker = CrossEncoderReranker(model=local_model, top_n=5)
local_reranking = ContextualCompressionRetriever(
    base_compressor=local_reranker,
    base_retriever=wide_retriever,
)
```

CHAPTER 12

Conversational RAG & Evaluation

Real applications need multi-turn RAG where follow-up questions reference previous answers. This chapter builds a history-aware RAG chain and covers evaluation with RAGAS.

12.1 History-Aware Retrieval

■ conversational_rag.py

```

from langchain.chains import (
    create_history_aware_retriever,
    create_retrieval_chain,
)
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# Reformulate follow-up as standalone question
contextualise_prompt = ChatPromptTemplate.from_messages([
    ('system',
     'Given chat history and the latest question, reformulate it '
     'as a standalone question. Do NOT answer -- just reformulate.'),
    MessagesPlaceholder('chat_history'),
    ('human', '{input}'),
])
history_aware_retriever = create_history_aware_retriever(
    llm, retriever, contextualise_prompt
)

# QA chain with history context
qa_prompt = ChatPromptTemplate.from_messages([
    ('system',
     'Use context to answer. Say so if answer not in context.\n\n'
     'Context:\n{context}'),
    MessagesPlaceholder('chat_history'),
    ('human', '{input}'),
])
rag_chain = create_retrieval_chain(
    history_aware_retriever,
    create_stuff_documents_chain(llm, qa_prompt),
)

# Add memory
store = {}
def get_history(session_id):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

conv_rag = RunnableWithMessageHistory(
    rag_chain, get_history,
    input_messages_key='input',
    history_messages_key='chat_history',
    output_messages_key='answer',
)

cfg = {'configurable': {'session_id': 's1'}}
r1 = conv_rag.invoke({'input': 'What is task decomposition?'}, config=cfg)
print(r1['answer'])

# 'it' refers to task decomposition -- history handles the context
r2 = conv_rag.invoke(
    {'input': 'What are the methods for doing it?'}, config=cfg
)
print(r2['answer'])

```

12.2 RAGAS Evaluation

Metric	What It Measures
--------	------------------

Faithfulness	Does answer contain only facts from context? (anti-hallucination)
Answer Relevancy	Is the answer relevant to the question? (anti-padding)
Context Precision	Are retrieved chunks actually useful? (retrieval precision)
Context Recall	Did retrieval find all necessary information? (retrieval recall)

■ ragas_evaluation.py

```
# pip install ragas
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall,
)
from datasets import Dataset

eval_data = {
    'question': [
        'What is task decomposition?',
        'What memory types do agents use?',
    ],
    'answer': [
        'Task decomposition breaks complex tasks into sub-tasks.',
        'Agents use in-context, external, and parametric memory.',
    ],
    'contexts': [
        ['Task decomposition involves breaking down...'],
        ['Memory in agents includes short-term...'],
    ],
    'ground_truth': [
        'Task decomposition is the process of breaking down a task.',
        'LLM agents use in-context storage and external databases.',
    ],
}

result = evaluate(
    Dataset.from_dict(eval_data),
    metrics=[
        faithfulness, answer_relevancy,
        context_precision, context_recall,
    ],
)
print(result.to_pandas())
# faithfulness  answer_relevancy  context_precision  context_recall
# 0.92          0.88          0.85          0.90
```

PART V

Agents & Tools

Agents are LLMs that decide what actions to take. Tools give them the ability to interact with the real world. LangGraph enables stateful, multi-step, multi-agent orchestration with full execution control.

CHAPTER 13

Chains — Legacy & Modern Patterns

Legacy chain classes are common in existing codebases. This chapter covers LLMChain, SequentialChain, and map-reduce alongside their modern LCEL equivalents so you can read old code and write new code confidently.

13.1 LLMChain (Legacy)

■ llmchain_legacy_vs_lcel.py

```
# Legacy -- still works, prefer LCEL for new code
from langchain.chains import LLMChain
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
prompt = ChatPromptTemplate.from_template('Capital of {country}?')

# Legacy LLMChain
chain = LLMChain(llm=llm, prompt=prompt)
result = chain.invoke({'country': 'France'})
print(result['text']) # 'Paris'

# LCEL equivalent (preferred for all new code)
lcel_chain = prompt | llm | StrOutputParser()
print(lcel_chain.invoke({'country': 'Japan'})) # 'Tokyo'
```

13.2 Map-Reduce for Long Documents

■ map_reduce_summarise.py

```
from langchain.chains.summarize import load_summarize_chain
from langchain_openai import ChatOpenAI
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
pages = PyPDFLoader('long_paper.pdf').load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=2000, chunk_overlap=200
).split_documents(pages)

# chain_type options:
# 'stuff' -- all docs in one prompt (short docs only)
# 'map_reduce' -- summarise each chunk, then combine
# 'refine' -- iteratively improve answer chunk by chunk
chain = load_summarize_chain(llm, chain_type='map_reduce')
result = chain.invoke({'input_documents': chunks})
print(result['output_text'])

# Custom map and combine prompts
map_prompt = ChatPromptTemplate.from_template(
    'Summarise in 3 bullet points:\n\n{text}'
)
combine_prompt = ChatPromptTemplate.from_template(
    'Combine these summaries into a final summary:\n\n{text}'
)
custom_chain = load_summarize_chain(
    llm, chain_type='map_reduce',
    map_prompt=map_prompt,
    combine_prompt=combine_prompt,
)
```

CHAPTER 14

Agent Fundamentals

Agents are LLMs that reason about what action to take next, execute that action using a tool, observe the result, and repeat until they produce a final answer. This chapter covers ReAct and the OpenAI Tools agent.

14.1 The ReAct Loop

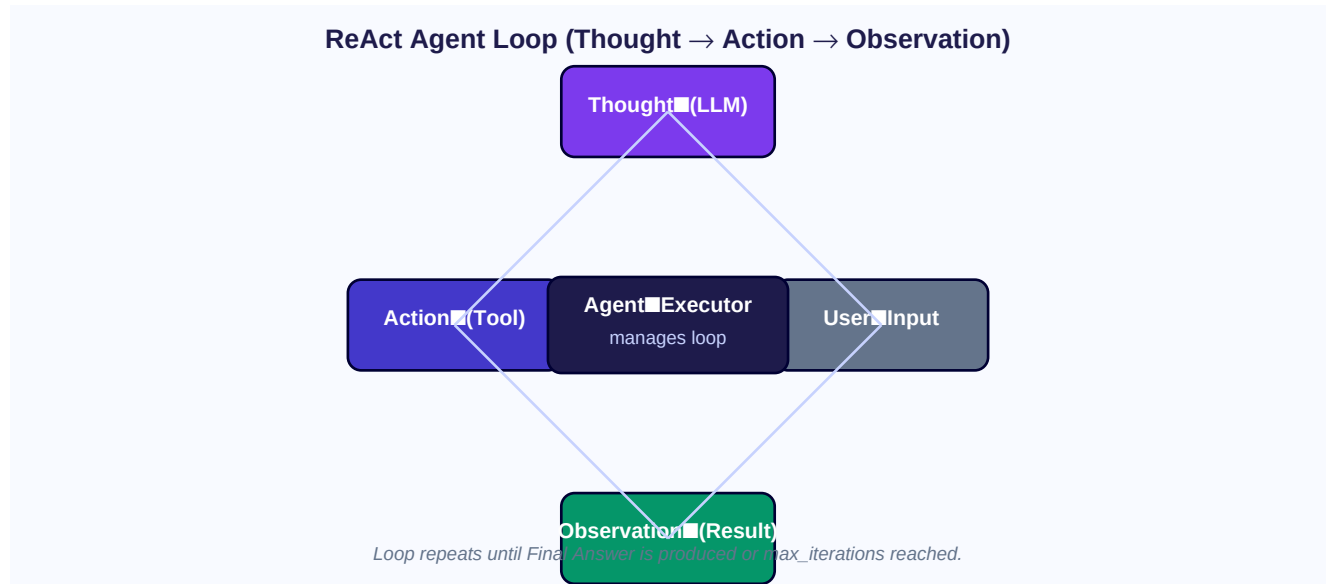


Figure 14.1 — ReAct Agent: Thought → Action → Observation loop

■ Key Concept

Key difference: chains have a fixed execution path set at code-write time. Agents have a dynamic path determined at runtime by the LLM.

14.2 ReAct Agent

■ react_agent.py

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_community.tools import DuckDuckGoSearchRun, WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper
from langchain import hub

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
tools = [
    DuckDuckGoSearchRun(),
    WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper()),
]

react_prompt = hub.pull('hwchase17/react')
agent = create_react_agent(llm=llm, tools=tools, prompt=react_prompt)

executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    max_iterations=10,
    max_execution_time=60,
    handle_parsing_errors=True,
    return_intermediate_steps=True,
)

result = executor.invoke({'input': 'Who founded LangChain and when?'})
print('Answer:', result['output'])

# Inspect reasoning trace
for step in result['intermediate_steps']:
    action, observation = step
    print(f'Tool: {action.tool}')
    print(f'Input: {action.tool_input}')
    print(f'Observation: {str(observation)[:80]}')
```

14.3 OpenAI Tools Agent

■ openai_tools_agent.py

```
from langchain.agents import create_openai_tools_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.tools import tool

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

@tool
def multiply(a: float, b: float) -> float:
    '''Multiply two numbers. Use when asked for a product.'''
    return a * b

@tool
def get_weather(city: str) -> str:
    '''Get current weather for a city. Returns temperature and conditions.'''
    return f'The weather in {city} is 25 degrees C and sunny.'

tools = [multiply, get_weather]
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant. Use tools when needed.'),
    MessagesPlaceholder('chat_history', optional=True),
    ('human', '{input}'),
    MessagesPlaceholder('agent_scratchpad'),
])

agent = create_openai_tools_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

print(executor.invoke({'input': '17 times 42?'})['output'])
print(executor.invoke({'input': 'Weather in Chennai?'})['output'])
```

CHAPTER 15

Tools & Toolkits

Tools are functions agents can call. The name and description are what the LLM sees when deciding which tool to use — they must be precise. This chapter covers custom tools, toolkits, SQL agent, and error handling.

■ Warning

Write tool descriptions as if explaining to a junior developer: what it does, what inputs it takes, what it returns, and when NOT to use it.

15.1 @tool Decorator

■ custom_tool.py

```
from langchain_core.tools import tool

@tool
def calculate_bmi(weight_kg: float, height_m: float) -> str:
    '''Calculate Body Mass Index (BMI).
    Args:
        weight_kg: Body weight in kilograms.
        height_m: Height in metres.
    Returns:
        BMI value and classification string.
    Use when the user asks about BMI or weight status.
    Do NOT use for calorie calculations.
    '''
    bmi = weight_kg / (height_m ** 2)
    if bmi < 18.5: cat = 'Underweight'
    elif bmi < 25.0: cat = 'Normal weight'
    elif bmi < 30.0: cat = 'Overweight'
    else: cat = 'Obese'
    return f'BMI: {bmi:.1f} ({cat})'

print(calculate_bmi.name) # 'calculate_bmi'
print(calculate_bmi.description) # the docstring
print(calculate_bmi.args_schema.schema()) # JSON schema

# Test directly
print(calculate_bmi.invoke({'weight_kg': 70, 'height_m': 1.75}))
# -> 'BMI: 22.9 (Normal weight)'
```

15.2 StructuredTool

■ structured_tool.py

```

from langchain_core.tools import StructuredTool
from pydantic import BaseModel, Field
from typing import Literal

class DBQueryInput(BaseModel):
    table: str = Field(description='Table name')
    column: str = Field(description='Column to filter')
    value: str = Field(description='Filter value')
    operation: Literal['eq', 'gt', 'lt', 'like'] = Field(default='eq')
    limit: int = Field(default=10)

def query_database(table, column, value, operation='eq', limit=10):
    '''Query the company database. Use for employee, product, or sales data.'''
    return (
        f'SELECT * FROM {table} WHERE {column} {operation} {value!r} LIMIT {limit}'
    )

db_tool = StructuredTool.from_function(
    func=query_database,
    name='query_database',
    description='Query the internal company database. Returns matching rows.',
    args_schema=DBQueryInput,
)

print(db_tool.invoke({
    'table': 'employees',
    'column': 'department',
    'value': 'Engineering',
}))

```

15.3 SQL Agent

■ sql_agent.py

```

from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import SQLDatabaseToolkit
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain import hub

db = SQLDatabase.from_uri('sqlite:///chinook.db')
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

agent = create_react_agent(
    llm, tools, hub.pull('hwchase17/react')
)
executor = AgentExecutor(
    agent=agent, tools=tools, verbose=True,
    handle_parsing_errors=True,
)

result = executor.invoke({
    'input': 'Which artist has the most albums? List the top 5.'
})
print(result['output'])

```

15.4 Tool Error Handling

■ tool_error_handling.py

```
from langchain_core.tools import tool, ToolException

@tool
def safe_divide(numerator: float, denominator: float) -> float:
    '''Divide numerator by denominator. Do not call with denominator=0.'''
    if denominator == 0:
        raise ToolException(
            'Cannot divide by zero. Provide a non-zero denominator.'
        )
    return numerator / denominator

# handle_tool_error options:
#   True      -> agent sees error message and can retry
#   False     -> exception propagates (crashes agent)
#   callable  -> custom error handler function

def custom_error_handler(error: ToolException) -> str:
    return f'Tool error: {str(error)}. Please adjust inputs and retry.'

# Attach error handler
safe_divide.handle_tool_error = custom_error_handler

# Or set at AgentExecutor level
from langchain.agents import AgentExecutor
executor = AgentExecutor(
    agent=agent,
    tools=[safe_divide],
    handle_parsing_errors=True, # recover from malformed LLM output
)
```

CHAPTER 16

LangGraph & Multi-Agent Systems

LangGraph is a library for building stateful, cyclical, multi-agent workflows as graphs. It gives precise control over execution, supports human-in-the-loop interrupts, and enables supervisor patterns.

16.1 AgentExecutor vs LangGraph

Feature	AgentExecutor	LangGraph
Execution	Simple while-loop	Directed graph: nodes + edges
State	Black-box internal	Explicit typed TypedDict
Cycles	Fixed loop only	Arbitrary cycles and conditional branches
Human-in-loop	Not natively supported	interrupt_before / interrupt_after
Persistence	No built-in	Checkpointers (SQLite, Redis)
Multi-agent	Shallow nesting via tools	Full supervisor + specialist graphs

16.2 Core Concepts: State, Nodes, Edges

■ langgraph_state_nodes_edges.py

```
# pip install langgraph
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated, Sequence
import operator

# 1. State -- the data that flows through the graph
class AgentState(TypedDict):
    messages: Annotated[Sequence[str], operator.add] # append-only list
    next_action: str
    iteration: int

# 2. Nodes -- functions: state -> partial state update
def researcher(state: AgentState) -> dict:
    return {
        'messages': ['Research complete.'],
        'iteration': state['iteration'] + 1,
    }

def writer(state: AgentState) -> dict:
    return {'messages': ['Draft written.'], 'next_action': 'done'}

def router(state: AgentState) -> str:
    if state['next_action'] == 'done' or state['iteration'] >= 3:
        return 'end'
    return 'write'

# 3. Build graph
graph = StateGraph(AgentState)
graph.add_node('researcher', researcher)
graph.add_node('writer', writer)
graph.set_entry_point('researcher')
graph.add_conditional_edges(
    'researcher', router,
    {'write': 'writer', 'end': END},
)
graph.add_edge('writer', END)

# 4. Compile and run
app = graph.compile()
result = app.invoke({
    'messages': ['Start'], 'next_action': '', 'iteration': 0
})
print(result['messages'])
```

16.3 Prebuilt ReAct Agent in LangGraph

■ langgraph_prebuilt_agent.py

```

from langgraph.prebuilt import create_react_agent
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

llm = ChatOpenAI(model='gpt-4o-mini')

@tool
def calculate(expression: str) -> str:
    '''Evaluate a Python math expression safely.
    Example: calculate("2 ** 10") returns '1024'
    '''
    try:
        return str(eval(expression, {'__builtins__': {}}, {}))
    except Exception as e:
        return f'Error: {e}'

agent = create_react_agent(llm, [calculate])

# Stream execution events node by node
for event in agent.stream({
    'messages': [('human', 'What is 2 to the power of 10?')]
}):
    for key, val in event.items():
        print(f'--- Node: {key} ---')
        if 'messages' in val:
            print(val['messages'][-1])

```

16.4 Persistence with Checkpointers

■ langgraph_checkpoint.py

```

from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

@tool
def add(a: int, b: int) -> int:
    '''Add two numbers together.'''
    return a + b

llm = ChatOpenAI(model='gpt-4o-mini')

with SqliteSaver.from_conn_string(':memory:') as checkpointer:
    agent = create_react_agent(llm, [add], checkpointer=checkpointer)
    config = {'configurable': {'thread_id': 'thread_001'}}

    r1 = agent.invoke(
        {'messages': [('human', 'Add 15 and 27')]}), config=config
    print(r1['messages'][-1].content) # 42

    # Agent remembers thread -- 'that result' refers to 42
    r2 = agent.invoke(
        {'messages': [('human', 'Now add 10 to that result.')]}),
        config=config,
    )
    print(r2['messages'][-1].content) # 52

```

PART VI

Production & Mastery

Build observable, testable, secure, and scalable LangChain applications. LangSmith tracing, caching strategies, LangServe deployment, security best practices, and a complete 60-question interview preparation guide.

CHAPTER 17

Callbacks & LangSmith Observability

You cannot improve what you cannot measure. LangSmith gives complete visibility into every LLM call, chain step, tool invocation, and token spent — across development, evaluation, and production monitoring.

17.1 Custom Callback Handler

■ custom_callback.py

```
from langchain_core.callbacks import BaseCallbackHandler
from langchain_core.outputs import LLMResult
import time

class LatencyTracker(BaseCallbackHandler):
    def __init__(self):
        self._start = None
        self.calls = []

    def on_llm_start(self, serialized, prompts, **kwargs):
        self._start = time.time()
        print(f'LLM started. Prompt: {len(prompts[0])} chars')

    def on_llm_end(self, response: LLMResult, **kwargs):
        latency = time.time() - self._start
        usage = response.llm_output.get('token_usage', {})
        self.calls.append({'latency_s': latency, 'tokens': usage})
        print(f'Done in {latency:.2f}s | Tokens: {usage}')

    def on_llm_error(self, error, **kwargs):
        print(f'LLM error: {error}')

from langchain_openai import ChatOpenAI
tracker = LatencyTracker()
llm = ChatOpenAI(model='gpt-4o-mini')

result = llm.invoke(
    'What is LangChain?',
    config={'callbacks': [tracker]}
)
print(f'Tracked {len(tracker.calls)} call(s)')
```

17.2 LangSmith Auto-Tracing

■ langsmith_tracing.py

```
# Just set these env vars -- zero code changes needed
import os
os.environ['LANGCHAIN_TRACING_V2'] = 'true'
os.environ['LANGCHAIN_API_KEY'] = 'ls__your_api_key'
os.environ['LANGCHAIN_PROJECT'] = 'production-rag'

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

chain = (
    ChatPromptTemplate.from_template('Explain {topic} in 2 sentences.')
    | ChatOpenAI(model='gpt-4o-mini')
    | StrOutputParser()
)

# Automatically traced in LangSmith dashboard
result = chain.invoke({'topic': 'RAG'})

# Add metadata to traces for filtering
result = chain.invoke(
    {'topic': 'agents'},
    config={
        'tags': ['production', 'v2'],
        'metadata': {'user_id': 'u123'},
        'run_name': 'explain-chain',
    }
)

# Manual tracing with @traceable
from langsmith import traceable

@traceable(name='my-pipeline', tags=['custom'])
def my_pipeline(topic: str) -> str:
    return chain.invoke({'topic': topic})
```

17.3 LangSmith Evaluation

■ langsmith_evaluation.py

```
from langsmith import Client
from langsmith.evaluation import evaluate, LangChainStringEvaluator
from langchain_openai import ChatOpenAI

client = Client()

# Create an evaluation dataset
examples = [
    {'inputs': {'question': 'What is LangChain?'},
     'outputs': {'answer': 'LangChain is a framework for LLM apps.'}},
    {'inputs': {'question': 'What is RAG?'},
     'outputs': {'answer': 'RAG combines retrieval with generation.'}},
]

dataset = client.create_dataset('RAG Eval Set')
client.create_examples(
    inputs=[e['inputs'] for e in examples],
    outputs=[e['outputs'] for e in examples],
    dataset_id=dataset.id,
)

# Target function to evaluate
def predict(inputs: dict) -> dict:
    answer = chain.invoke({'topic': inputs['question']})
    return {'answer': answer}

# Run evaluation
qa_eval = LangChainStringEvaluator(
    'qa', config={'llm': ChatOpenAI(model='gpt-4o')}
)
results = evaluate(
    predict,
    data='RAG Eval Set',
    evaluators=[qa_eval],
    experiment_prefix='rag-v1',
)
print(results.to_pandas()[['inputs.question', 'feedback.correctness']])
```

CHAPTER 18

Caching, Deployment & Security

Production LangChain applications need low latency (caching), stable HTTP APIs (LangServe), and protection against prompt injection and data leakage.

18.1 Caching Strategies

■ caching_strategies.py

```
# Exact LLM response cache
from langchain.globals import set_llm_cache
from langchain_community.cache import InMemoryCache, SQLiteCache

set_llm_cache(InMemoryCache())          # dev
set_llm_cache(SQLiteCache('.langchain.db')) # staging

# Redis distributed cache (production)
import redis
from langchain_community.cache import RedisCache
set_llm_cache(RedisCache(
    redis=redis.Redis.from_url('redis://localhost:6379')
))

# Semantic cache -- hits on similar (not identical) queries
from langchain_community.cache import RedisSemanticCache
from langchain_openai import OpenAIEmbeddings

set_llm_cache(RedisSemanticCache(
    redis_url='redis://localhost:6379',
    embedding=OpenAIEmbeddings(),
    score_threshold=0.2,
))

# Embedding cache -- avoid re-embedding same text
from langchain.embeddings import CacheBackedEmbeddings
from langchain_community.storage import LocalFileStore
from langchain_openai import OpenAIEmbeddings

base = OpenAIEmbeddings(model='text-embedding-3-small')
store = LocalFileStore('./emb_cache/')
cached = CacheBackedEmbeddings.from_bytes_store(
    base, store, namespace=base.model
)
```

18.2 LangServe Deployment

■ server.py

```
# pip install 'langserve[all]' fastapi uvicorn

# server.py
from fastapi import FastAPI
from langserve import add_routes
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

app = FastAPI(title='LangChain API', version='1.0')
chain = (
    ChatPromptTemplate.from_template(
        'Explain {topic} in simple terms.'
    )
    | ChatOpenAI(model='gpt-4o-mini')
    | StrOutputParser()
)

# Auto-creates: /explain/invoke /explain/stream /explain/batch
add_routes(app, chain, path='/explain')

if __name__ == '__main__':
    import uvicorn
    uvicorn.run(app, host='0.0.0.0', port=8000)
```

■ client.py

```
# client.py -- call the LangServe endpoint
from langserve import RemoteRunnable

remote = RemoteRunnable('http://localhost:8000/explain')

# Works exactly like a local chain
print(remote.invoke({'topic': 'RAG'}))

# Streaming over HTTP
for chunk in remote.stream({'topic': 'LangGraph'}):
    print(chunk, end='', flush=True)

# Batch
results = remote.batch([
    {'topic': 'RAG'},
    {'topic': 'agents'},
])
```

18.3 Security Best Practices

✗ Important

Prompt injection, data leakage, and tool misuse are real threats. Security is non-negotiable in production LLM applications.

■ security_practices.py

```
# 1. Input validation with Pydantic
from pydantic import BaseModel, validator, Field

class SafeInput(BaseModel):
    question: str = Field(max_length=500)

    @validator('question')
    def no_injection(cls, v):
        blocked = ['ignore previous', 'disregard', 'you are now']
        for phrase in blocked:
            if phrase in v.lower():
                raise ValueError(f'Blocked: {phrase}')
        return v

# 2. PII redaction before embedding
import re

def redact_pii(text: str) -> str:
    text = re.sub(r'[\w.-]+@[ \w.-]+\.\w+', '[EMAIL]', text)
    text = re.sub(r'\b\d{3}[-.]?\d{3}[-.]?\d{4}\b', '[PHONE]', text)
    text = re.sub(r'\b\d{3}-\d{2}-\d{4}\b', '[SSN]', text)
    return text

for doc in documents:
    doc.page_content = redact_pii(doc.page_content)

# 3. Rate limiting with SlowAPI
# pip install slowapi
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
# @app.get('/chat')
# @limiter.limit('10/minute')
# async def chat_endpoint(request: Request, q: str): ...
```

CHAPTER 19

Interview Preparation Guide

60 curated questions spanning conceptual understanding, system design, debugging, and coding — covering every major topic in this book.

19.1 Conceptual Questions

✓ Tip

Always explain the 'why', not just the 'what'. Interviewers want to see you understand trade-offs, not just API signatures.

Q1 What is the Runnable protocol and why does every LangChain component implement it?

✓ Key Answer: Every component implements `invoke/stream/batch/ainvoke`. This gives uniform composability: any component can be piped into any other with `|`. Streaming and async work transparently end-to-end automatically.

Q2 Difference between a ChatModel and an LLM in LangChain?

✓ Key Answer: LLM: string in, string out (completion-style, legacy). ChatModel: `list[BaseMessage]` in, `AIMessage` out (modern). All frontier models today are ChatModels.

Q3 Explain LCEL and the pipe operator `|`.

✓ Key Answer: LCEL = Runnable protocol + `|` operator. `chain = prompt | llm | parser` -- each component's output feeds the next. Streaming works end-to-end. The composed chain inherits all Runnable methods.

Q4 What is RunnableParallel and when would you use it?

✓ Key Answer: Runs multiple branches concurrently on the same input and merges into a dict. Use when you need independent operations (e.g., retrieve AND pass through the question) before the next chain step.

Q5 How does RunnableWithMessageHistory work?

✓ Key Answer: Before `invoke`: loads history from session store, injects into input. After `invoke`: saves new human+AI turn back to store. The chain stays stateless; history is managed externally.

Q6 Difference between stuff, map-reduce, and refine document strategies?

✓ Key Answer: Stuff: all docs in one prompt (fast, limited by context window). Map-reduce: summarise each chunk then aggregate (handles long docs). Refine: iteratively improve answer chunk by chunk (most expensive, highest quality).

Q7 When would you use SemanticChunker over RecursiveCharacterTextSplitter?

✓ Key Answer: RecursiveCharacterTextSplitter: fast, good default. SemanticChunker: use when document sections shift topics without consistent separators. Slower and costs API calls.

Q8 What is HyDE and when does it improve RAG?

✓ **Key Answer:** Generate a hypothetical answer using LLM, embed that answer, search with it. Works because the hypothetical answer is in the same embedding space as documents. Improves recall for factual questions where query and document embeddings are dissimilar.

Q9 Explain the difference between AgentExecutor and LangGraph.

✓ **Key Answer:** AgentExecutor: simple while-loop, black-box state, no built-in persistence. LangGraph: full graph with typed state, conditional edges, cycles, human-in-the-loop, checkpointed persistence. Use LangGraph for anything complex.

Q10 What is the LangChain Indexing API and why is it needed?

✓ **Key Answer:** Tracks indexed documents by content hash via RecordManager. On re-ingestion it only re-embeds changed documents and removes orphaned vectors. Without it, re-running indexing creates duplicates and wastes API cost.

19.2 System Design Questions

✓ Tip

Include: architecture with data flow, component choices with reasoning, failure modes, cost estimate, and monitoring strategy.

SD1 Design a production RAG system for a 10-million page legal corpus. Include: chunking, embedding model, vector store, retrieval, latency targets.

SD2 Design a multi-tenant chatbot where 10,000 customers have isolated memory and private document corpora. How do you prevent data leakage?

SD3 Design a customer support agent that queries your DB, searches a knowledge base, and escalates to a human when it cannot answer. How do you handle handoff?

SD4 Your RAG system produces factually wrong answers. Walk through your debugging process — which RAGAS metrics do you check first?

SD5 Design a pipeline to sync a vector store with a database receiving 1,000 document updates per day. Handle additions, updates, and deletions.

SD6 Build a code generation agent that writes, executes, and debugs Python. What safety measures do you put in place for code execution?

19.3 Debugging Questions

DBG1: Retriever returns 4 relevant chunks but the final answer is wrong.

✓ **Approach:** Print the chunks — are they relevant? Is {context} injected correctly? Is the LLM ignoring context and hallucinating? Enable LangSmith tracing to see the exact prompt sent to the LLM.

DBG2: Agent stuck calling the same tool in an infinite loop.

✓ **Approach:** Set max_iterations and max_execution_time. Enable verbose=True. Common causes: tool returns ambiguous output; description too vague; LLM not recognising the Final Answer stopping condition.

DBG3: LCEL chain works with .invoke() but hangs with .astream().

✓ **Approach:** A component is not truly async. Check for blocking DB/IO calls inside an async context. Use loop.run_in_executor() for blocking calls. Ensure RunnableLambda coroutines are defined with async def.

DBG4: Embedding costs are 50x the estimate.

✓ **Approach:** You are re-embedding all documents on every run. Fix with LangChain Indexing API + RecordManager (hash-based dedup). Add CacheBackedEmbeddings for identical chunks.

DBG5: PydanticOutputParser fails 30% of the time.

✓ **Approach:** (1) Add OutputFixingParser to auto-retry. (2) Switch to llm.with_structured_output(Model) which uses tool-calling — far more reliable. (3) Set temperature=0.

DBG6: Conversational RAG gives wrong answer when user says 'it' referring to previous topic.

✓ **Approach:** Question condensation step failed to replace 'it' with the actual topic from history. Check that create_history_aware_retriever receives full chat_history. Inspect the reformulated question in LangSmith traces.

19.4 Coding Patterns to Master

Pattern	Key Components to Demonstrate
Basic LCEL Chain	ChatPromptTemplate ChatOpenAI StrOutputParser + .stream()
Structured Extraction	with_structured_output(PydanticModel) -- no format instructions needed
Naive RAG End-to-End	Load -> Split -> FAISS -> Retrieve -> Prompt -> LLM -> Answer
Conversational Chatbot	ChatPromptTemplate + MessagesPlaceholder + RunnableWithMessageHistory
Custom Tool + Agent	@tool decorator + create_openai_tools_agent + AgentExecutor
Parallel Branches	RunnableParallel with two independent analysis chains on same input
Conditional Router	RunnableBranch routing to specialist chains by keyword or embedding
Map-Reduce Summary	load_summarize_chain(llm, chain_type='map_reduce') on split PDF

LangGraph Agent	StateGraph + ToolNode + conditional edges + compile + .stream()
Hybrid RAG Retriever	EnsembleRetriever: BM25 + FAISS with RRF weights

You have reached the end of **Mastering LangChain** by **Sabarinathan J**. The fastest path to mastery is: read a chapter, then build the project from scratch without looking at the code. Break it. Fix it. Add a feature. Repeat for all 19 chapters. Use LangSmith to trace everything you build and study the traces as carefully as you study the code.

— *Sabarinathan J*